Artificial Intelligence Technology and Application

Machine Learning Lab Guide

Student Version

Huawei Technologies CO., LTD.

About This Document

Description

This lab guide comprises the following five experiments:

- Experiment 1 - Linear regression. The Python tool package scikit-learn is used to implement a simple linear regression algorithm.
- Experiment 2 - Linear regression expansion. The Python basic tool package numpy is used to implement algorithms such as linear regression and gradient descent from scratch.
- Experiment 3 - Logistic regression. The logistic regression algorithm in the tool package is used to implement simple classification.
- Experiment 4 - Decision tree. Trainees will construct a decision tree to predict the weather, and visualize the decision tree.
- Experiment 5 - K-means clustering algorithm.

Background Knowledge Required

This course is for Huawei's professional certification. Trainees are expected to meet the following requirements:

- Have basic knowledge of Python programming.
- Have basic math skills, including linear algebra and probability theory.

Lab Environment Overview

The lab environment is Python 3.7. For details about how to set up the environment, see the Environment Setup Guide.

Datasets used in this lab can be obtained at https://certification-data.obs.cn-north-4.myhuaweicloud.com/ENG/HCIA-AI/V3.5/chapter2/ML.zip

Contents

About This Document 2
Description 2
Background Knowledge Required 2
Lab Environment Overview 2
1 Implementation of Common Machine Learning Algorithms 4
1.1 Introduction 4
1.1.1 About This Lab 4
1.1.2 Objectives 4
1.2 Code Implementation 4
1.2.1 Linear Regression 4
1.2.2 Linear Regression Implementation (Expansion Experiment) 6
1.2.3 Logistic Regression 9
1.2.4 Decision Tree 11
1.2.5 K-means Algorithm Implementation 13
1.3 Question 18
1.4 Summary 18

1 Implementation of Common Machine Learning Algorithms

1.1 Introduction

1.1.1 About This Lab

This lab introduces common machine learning algorithms to help you better understand their functions and usages. Specifically, it will explain how to build a linear regression algorithm from scratch and implement the decision tree and K-means clustering based on scikit-learn.

1.1.2 Objectives

- Build a linear regression algorithm from scratch.
- Master the use of classification and regression algorithms.
- Master the use of the V clustering algorithm.
- Master the implementation process of machine learning algorithms.

1.2 Code Implementation

1.2.1 Linear Regression

Step 1 Import dependent packages.

Input:

```
from sklearn.linear_model import LinearRegression # Import the linear regression model
import matplotlib.pyplot as plt # The plotting library
import numpy as np
```

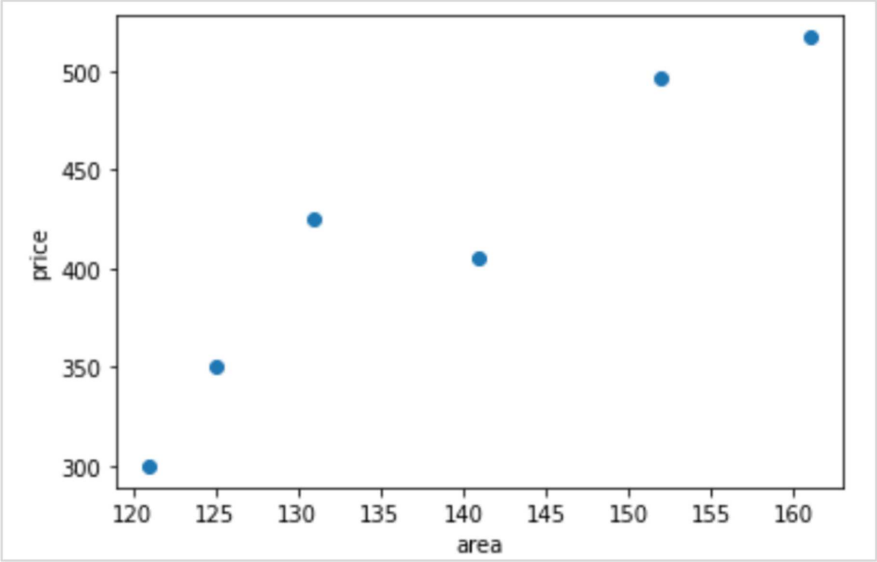
Step 2 Build and visualize a house price dataset.

Input:

```
x = np.array([121, 125, 131, 141, 152, 161]).reshape(-1,1) # x denotes the house area as a feature.
y = np.array([300, 350, 425, 405,496,517]) # y denotes the house price.
plt.scatter(x,y)
plt.xlabel("area") # X axis indicates the area.
plt.ylabel("price") # Y axis indicates the price.
```

```
plt.show()
```

Output:



Step 3 Train the model.

Input:

```
#train the model
```

Step 4 Visualize the model.

Input:

```
# visualize the model
```

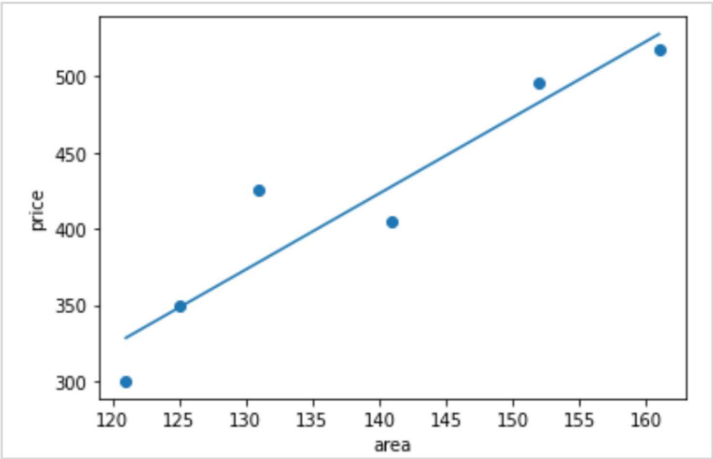
Output:

```
Slope: [4.98467124]
Intercept: -274.8769665187576
```

Input:

```
plt.scatter(x,y)
plt.xlabel("area") # X axis indicates the area.
plt.ylabel("price") # Y axis indicates the price.
plt.plot([x[0],x[-1]],[x[0]*w+b,x[-1]*w+b])
```

Output:



Step 5 Start a prediction task using the model.

Input:

```
testX = np.array([[130]])# A test sample with an area of 130
lr.predict(testX)
```

Output:

```
Array([373.13029447]) # The model predicts the house price of the sample.
```

1.2.2 Linear Regression Implementation (Expansion Experiment)

This experiment uses the **lr2_data.txt** dataset, which contains simulated house area and price data. To obtain the dataset, see the Lab Environment Setup.

Step 1 Import dependencies.

Input:

```
import numpy as np
import matplotlib.pyplot as plt
```

Step 2 Define the function for calculating gradients.

Input:

```
#define the function for calculating gradients
```

Step 3 Define the function for reading datasets.

Input:

```
def get_training_data(file_path):
    orig_data = np.loadtxt(file_path,skiprows=1) # Ignore the title in the first row of the dataset.
```

```
    cols = orig_data.shape[1]
    return (orig_data, orig_data[:, :cols - 1], orig_data[:, cols-1:])
```

Step 4 Define the function for initializing parameters.

Input:

```
# Initialize the θ array.
def init_theta(feature_count):
    return np.ones(feature_count).reshape(feature_count, 1)
```

Step 5 Define the function for implementing gradient descent.

Input:

```
def gradient_descending(X, y, theta, alpha):
    Jthetas= []   # Record the change trend of the cost function J(θ) to confirm the gradient descent is correct.
    # Calculate the loss function, which is equal to the square of the difference between the actual value and the predicted value: (y^i-h(x^i))^2
    Jtheta = (X.dot(theta)-y).T.dot(X.dot(theta)-y)
    index = 0
    gradient = generate_gradient(X, theta, y) # Calculate the gradient.
    while not np.all(np.absolute(gradient) <= 1e-5):   # End the calculation when the gradient is less than 0.00001.
        theta = theta - alpha * gradient
        gradient = generate_gradient(X, theta, y) # Calculate the new gradient.
        # Calculate the loss function, which is equal to the square of the difference between the actual value and the predicted value: (y^i-h(x^i))^2
        Jtheta = (X.dot(theta)-y).T.dot(X.dot(theta)-y)
        if (index+1) % 10 == 0:
            Jthetas.append((index, Jtheta[0]))   # Record the result every 10 calculations.
        index += 1
    return theta,Jthetas
```

Step 6 Define the function for visualizing the change curve of the loss function.

Input:

```
#define the function for visualizing the change curve of the loss function
#Plot the loss function change curve.
```

Step 7 Define the function for visualizing data points and the fitted curve.

Input:

```
# Plot the actual data points and the fitted curve.
def showlinercurve(theta, sample_training_set):
    x, y = sample_training_set[:, 1], sample_training_set[:, 2]
    z = theta[0] + theta[1] * x
    plt.scatter(x, y, color='b', marker='x',label="sample data")
```

```
    plt.plot(x, z, 'r', color="r",label="regression curve")
    plt.xlabel('x')
    plt.ylabel('y')
    plt.title('liner regression curve')
    plt.legend()
    plt.show()
```

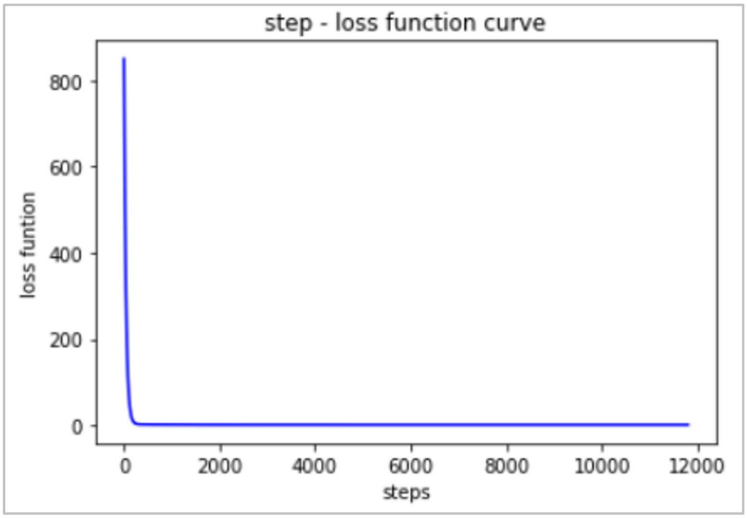
Step 8 Plot the final results.

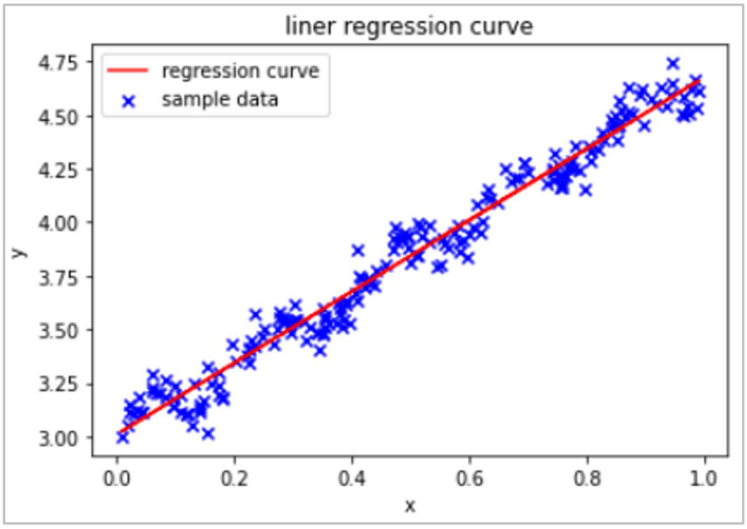
Input:

```
# Read the dataset.
training_data_include_y, training_x, y = get_training_data("./ML/02/lr2_data.txt")
# Obtain the numbers of samples and features, respectively.
sample_count, feature_count = training_x.shape
# Define the learning step α.
alpha = 0.01
# Initialize θ.
theta = init_theta(feature_count)
# Obtain the final parameter θ and cost.
result_theta,Jthetas = gradient_descending(training_x, y, theta, alpha)
# Display the parameter.
print("w:{}".format(result_theta[0][0]),"b:{}".format(result_theta[1][0]))
showJTheta(Jthetas)
showlinercurve(result_theta, training_data_include_y)
```

Output (ignore warnings):

```
w:3.0076279423997594 b:1.668677412281192
```





1.2.3 Logistic Regression

This experiment uses a custom dataset of house rent and area. The dataset is defined during the initial phase of the experiment.

Step 1 Import dependencies.

Input:

```
# Import StandardScaler from sklearn.preprocessing.
from sklearn.preprocessing import StandardScaler
# Import LogisticRegression from sklearn.linear_model.
from sklearn.linear_model import LogisticRegression
```

Step 2 Define the dataset.

Input:

```
# Each item in X denotes the rent and area.
# y indicates whether to rent the room (0: no; 1: yes).
X=[[2200,15],[2750,20],[5000,40],[4000,20],[3300,20],[2000,10],[2500,12],[12000,80],
    [2880,10],[2300,15],[1500,10],[3000,8],[2000,14],[2000,10],[2150,8],[3400,20],
    [5000,20],[4000,10],[3300,15],[2000,12],[2500,14],[10000,100],[3150,10],
    [2950,15],[1500,5],[3000,18],[8000,12],[2220,14],[6000,100],[3050,10]
  ]

y=[1,1,0,0,1,1,1,1,0,1,1,0,1,1,0,1,0,0,0,1,1,1,0,1,0,1,0,1,1,0]
```

Step 3 Preprocess data.

Standardize data to ensure that the variance of feature data in each dimension is 1 and the mean is 0. In this way, the prediction result is not dominated by large feature values of some dimensions.

Input:

```
ss = StandardScaler()
X_train = ss.fit_transform(X)
```

Display the standardized data.

Input:

```
print(X_train)
```

Output:

```
[[-0.60583897 -0.29313058]
 [-0.37682768 -0.09050576]
 [ 0.56003671  0.71999355]
 [ 0.14365254 -0.09050576]
 [-0.14781638 -0.09050576]
 [-0.68911581 -0.49575541]
 [-0.48092372 -0.41470548]
 [ 3.47472592  2.34099218]
 [-0.32269773 -0.49575541]
 [-0.56420055 -0.29313058]
 [-0.89730789 -0.49575541]
 [-0.27273163 -0.57680534]
 [-0.68911581 -0.33365555]
 [-0.68911581 -0.49575541]
 [-0.62665818 -0.57680534]
 [-0.10617796 -0.09050576]
 [ 0.56003671 -0.09050576]
 [ 0.14365254 -0.49575541]
 [-0.14781638 -0.29313058]
 [-0.68911581 -0.41470548]
 [-0.48092372 -0.33365555]
 [ 2.64195758  3.15149149]
 [-0.21027401 -0.49575541]
 [-0.29355084 -0.29313058]
 [-0.89730789 -0.69838024]
 [-0.27273163 -0.17155569]
 [ 1.80918923 -0.41470548]
 [-0.59751129 -0.33365555]
 [ 0.97642089  3.15149149]
 [-0.25191242 -0.49575541]]
```

Step 4 Fit the data.

Input:

```
# Use the fit method of LogisticRegression to train model parameters.
lr = LogisticRegression()
lr.fit(X_train, y)
```

Output:

```
LogisticRegression()
```

Step 5 Predict the data.

Input:

```
testX = [[2000,8]]
X_test = ss.transform(testX)
print("Value to be predicted: ",X_test)
label = lr.predict(X_test)
print("predicted label = ", label)
# Output the predicted probability.
prob = lr.predict_proba(X_test)
print("probability = ",prob)
```

Output:

```
Value to be predicted: [[-0.68911581 -0.57680534]]
predicted label =  [1]
probability =  [[0.41886952 0.58113048]]
```

1.2.4 Decision Tree

This experiment uses the **tennis.txt** dataset, which contains 14 samples. Each sample contains weather-related features and whether it is suitable for tennis.

Step 1 Import dependencies.

Input:

```
import pandas as pd
import numpy as np
from sklearn import tree
import pydotplus
```

Step 2 Define the function for generating a decision tree.

Input:

```
# Generate a decision tree.
```

Step 3 Define the function for saving the generated tree diagram.

Input:

```
def showtree2pdf(trainedTree,finename):
    dot_data = tree.export_graphviz(trainedTree, out_file=None) # Export the tree in Graphviz format.
    graph = pydotplus.graph_from_dot_data(dot_data)
    graph.write_pdf(finename)  # Save the tree diagram to the local machine in PDF format.
```

Step 4 Define the function for generating vectorized data.

In the function, **pd.Categorical(list).codes** obtains the sequence number list corresponding to the original data, so as to convert the categorical information into numeric information.

Input:

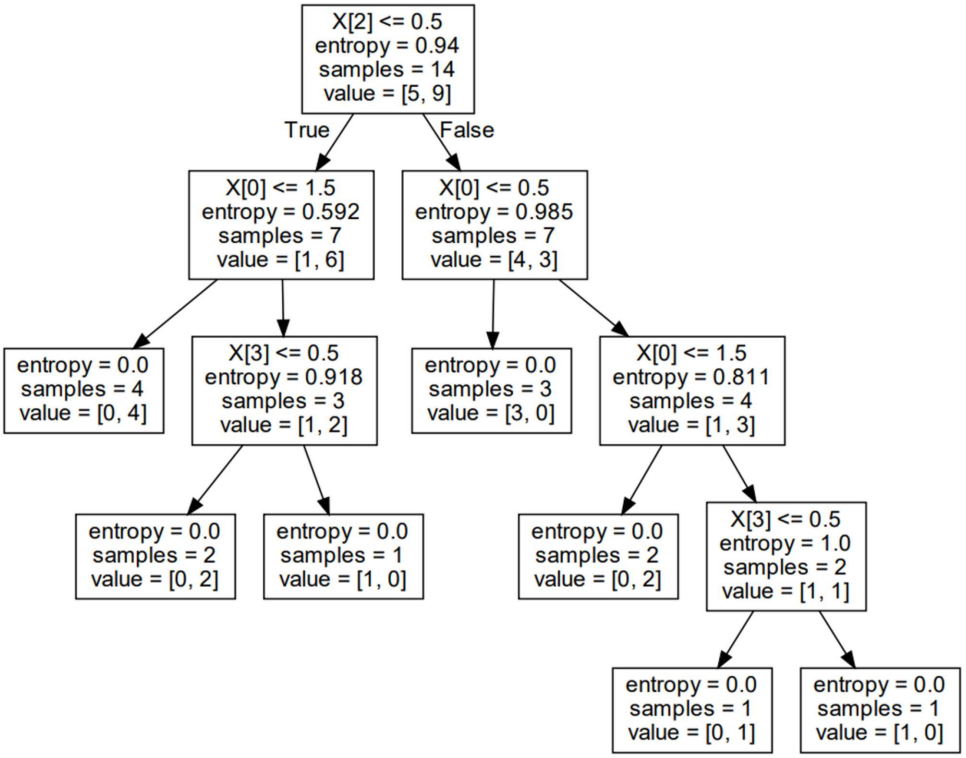
```
# define the function for generating vectorized data.
```

Step 5 Invoke the function for prediction.

Input:

```
data = pd.read_table("./ML/tennis.txt",header=None,sep='\t') # Read training data.
trainingvec=data2vectoc(data) # Vectorize data.
decisionTree=createTree(trainingvec) # Create a decision tree.
showtree2pdf(decisionTree,"tennis.pdf") # Plot the decision tree.
```

A decision tree diagram named **tennis.pdf** is generated on the local machine.



The file content is a visualized display of the decision tree. In the diagram, **X[2]** is the third feature variable (humidity); **X[0]** is the first feature variable (weather); **X[3]** is the fourth feature variable (wind); **entropy** is the entropy value of the node; and **samples** is the number of samples in the node, for example, 14 in the first node (root node) indicates the number of samples in the training set; and **value** indicates the numbers of

samples of different types, for example, in the root node, 5 indicates the number of "no" samples, and 9 indicates the number of "yes" samples.

Predict a new sample. Input:

```
testVec = [0,0,1,1] # Weather is sunny, temperature is low, humidity is high, and wind is strong.
print(decisionTree.predict(np.array(testVec).reshape(1,-1))) # Predict.
```

Output:

```
['N']
```

1.2.5 K-means Algorithm Implementation

Step 1 Import dependencies.

make_blobs is used to generate the dataset required for this experiment, **matplotlib** is used for data visualization, and **KMeans** is used for fitting data based on the K-means algorithm.

Input:

```
from sklearn.datasets import make_blobs
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
```

Step 2 Generate the dataset.

make_blobs() is used to generate a dataset for a clustering task by returning the generated dataset and labels. Note: the K-means algorithm is often used to process unlabeled datasets in production. The generated dataset contains labels only to facilitate understanding.

Input:

```
X, y = make_blobs(n_samples=500,n_features=2,centers=4,random_state=1)
```

n_samples is the number of samples, **n_features** is the number of features of each sample (the dimension of data), **centers** is the number of categories, and **random_state** is the seed for the random number generator. The seed ensures that data generated each time is the same.

Display the dimension information of the generated dataset.

Input:

```
# Display the dimension information of the generated dataset
```

Output:

```
Dimension of X is (500, 2)
Dimension of y is (500,).
```

The dataset contains 500 sample points, and each sample point contains two features.

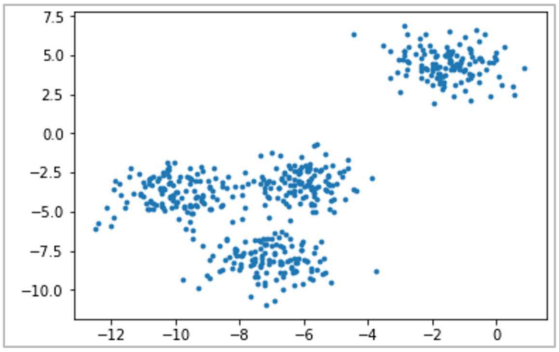
Step 3 Draw scatter graphs.

First, directly draw a scatter graph of the dataset. The labels generated during dataset generation are not considered for now. The graph contains four different clusters.

Input:

```
#draw scatter graphs
```

Output:

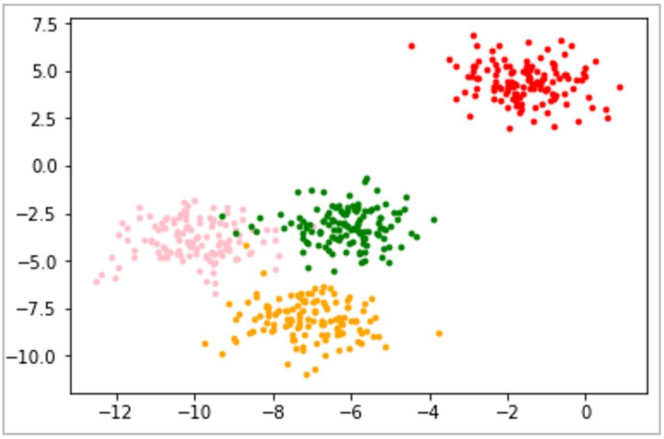


Next, draw a scatter graph where points in different colors correspond to the generated labels.

Input:

```
# draw a scatter graph where points in different colors correspond to the generated labels
```

Output:



Step 4 Perform k-means clustering.

Use **sklearn.cluster.KMeans** provided by scikit-learn to implement K-means clustering. First, aggregate the data samples into three types.

Input:

```
#Perform k-means clustering
```

View the label of each sample point after clustering.

Input:

```
# View the label of each sample point after clustering
```

Output:

```
[0 0 2 1 2 1 2 2 2 2 0 0 2 1 2 0 2 0 1 2 2 2 2 1 2 2 1 1 2 2 0 1 2 0 2 0 2
 2 0 2 2 2 1 2 2 0 2 2 1 1 1 2 2 2 0 2 2 2 2 2 1 1 2 2 1 2 0 2 2 2 0 2 2 0
 2 2 0 2 2 2 1 1 2 1 1 2 2 1 2 2 1 0 2 2 1 0 0 2 0 1 1 0 1 2 1 2 2 1 1 2 2
 0 1 2 1 2 1 2 1 2 2 0 0 2 2 2 1 0 0 2 1 2 2 2 2 0 1 2 1 1 2 0 2 1 1 1 2 2
 0 0 2 2 1 0 1 2 2 2 2 2 2 2 2 2 1 0 0 0 2 1 0 2 2 0 1 2 2 2 2 0 2 2 1 0 0
 2 2 0 0 2 1 1 0 0 2 1 2 0 0 1 0 2 1 2 2 0 2 2 0 2 2 2 2 0 2 2 2 1 2 1 2 0
 2 2 2 2 2 1 2 1 0 2 0 2 1 1 2 0 1 0 2 2 0 0 0 0 2 2 0 2 2 1 1 2 2 1 2 2 2
 1 2 1 2 2 1 2 0 0 2 2 2 2 1 1 2 1 2 0 1 0 1 0 0 1 0 1 1 2 2 2 2 2 2 2 0 1
 0 0 0 2 2 2 0 2 0 0 2 0 0 2 1 0 2 2 1 1 2 0 1 1 2 0 1 1 2 2 1 2 2 0 0 1 2
 0 2 1 1 2 2 2 0 2 1 1 2 1 1 1 1 0 0 2 1 2 2 0 1 2 1 2 1 2 2 2 1 2 2 0 1 0
 0 0 0 0 0 2 0 1 0 1 1 2 1 2 2 2 0 1 2 1 2 0 2 2 0 2 2 1 1 0 2 2 1 2 2 0 0
 2 0 2 2 0 2 0 2 1 0 1 2 2 1 2 2 1 0 2 1 1 2 2 2 2 0 1 0 2 1 0 0 0 2 1 2 0
 2 2 2 2 0 2 2 2 2 2 2 0 2 2 0 2 1 2 1 2 2 2 1 1 1 2 2 2 0 2 1 2 0 1 0 1 0
 2 1 1 0 2 2 0 2 2 2 0 2 1 2 2 0 0 0 2]
```

Print the centroid of each cluster.

Input:

```
centroid1 = cluster1.cluster_centers_
print(centroid1)
```

Output:

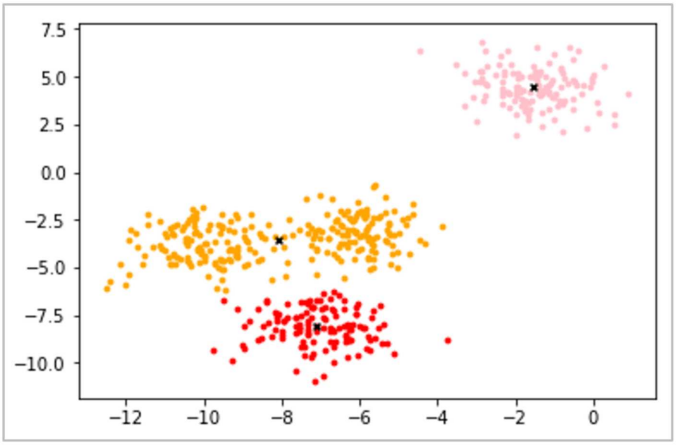
```
[[-7.09306648 -8.10994454]
 [-1.54234022  4.43517599]
 [-8.0862351  -3.5179868 ]]
```

Visualize the clustering result.

Input:

```
# Visualize the clustering result
```

Output:



The graph shows that the data samples are aggregated into three clusters, and the centroid of each cluster is represented by **x**.

Step 5 Perform k-means clustering again.

Next, perform the preceding steps to use k-means clustering to aggregate data samples into four types.

Input:

```
n_clusters = 4
cluster2 = KMeans(n_clusters=n_clusters,random_state=0).fit(X)
```

```
y_pred2 = cluster2.labels_
centroid2 = cluster2.cluster_centers_
print("Centroid: {}".format(centroid2))
```

Output:

```
Centroid: [[ -6.08459039   -3.17305983]
 [ -1.54234022    4.43517599]
 [ -7.09306648   -8.10994454]
 [-10.00969056   -3.84944007]]
```

Visualize the clustering result.

Input:

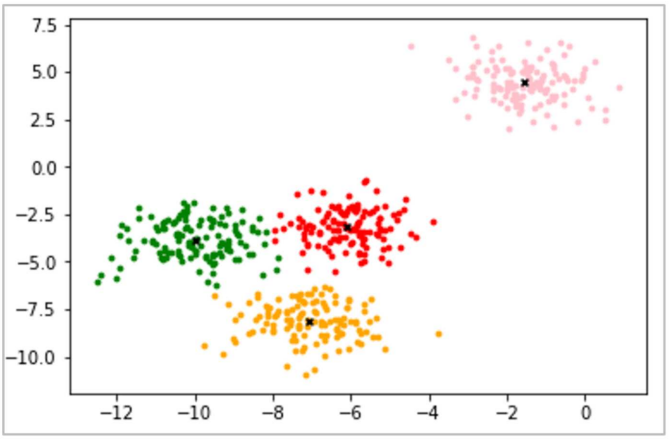
```
color = ["red","pink","orange","green"]

fig, ax1 = plt.subplots(1)

for i in range(n_clusters):
    ax1.scatter(X[y_pred2==i, 0], X[y_pred2==i, 1]
                ,marker='o' # Set the shape of the point to circle.
                ,s=8 # Set the size of the point.
                ,c=color[i]
               )

ax1.scatter(centroid2[:,0],centroid2[:,1]
            ,marker="x"
            ,s=15
            ,c="black")
plt.show()
```

Output:



The graph shows that the data samples are aggregated into four clusters, and the centroid of each cluster is represented by **x**. Compare the scatter graphs generated based on the original data and data aggregated into four types. It can be seen that many sample points are aggregated into wrong clusters.

1.3 Question

How to implement K-means from scratch using Python?

1.4 Summary

This lab introduces the machine learning implementation process, including data import, segmentation, standardization, as well as models and hyperparameters. Common machine learning algorithms are implemented based on scikit-learn to help you better understand how to build and use machine learning models.